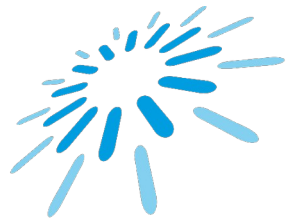




Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 04



TH Aschaffenburg
university of applied sciences

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00			Lecture Software Design International		
	Lunch Break				
14:00 - 15:30			Exercises Software Design International		
15:45 - 17:15					
17:30 - 19:00					

What is an ADR?

What role do quality requirements play in ADR?

Software Architecture: Communicating Architecture

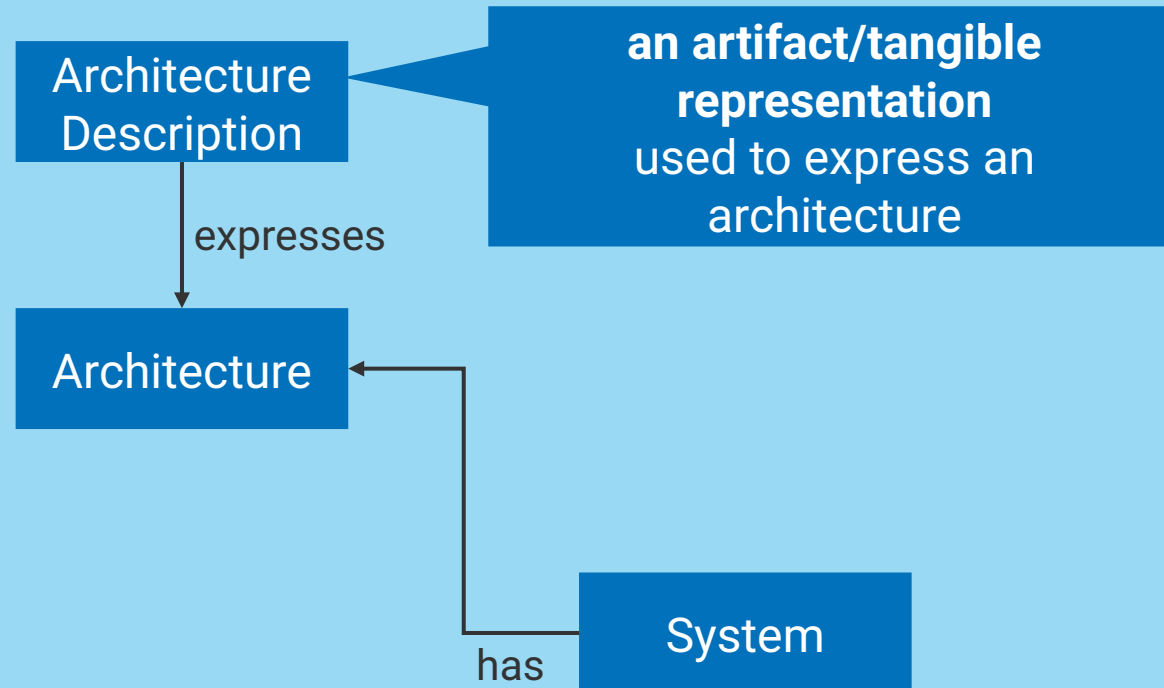


Architecture Description

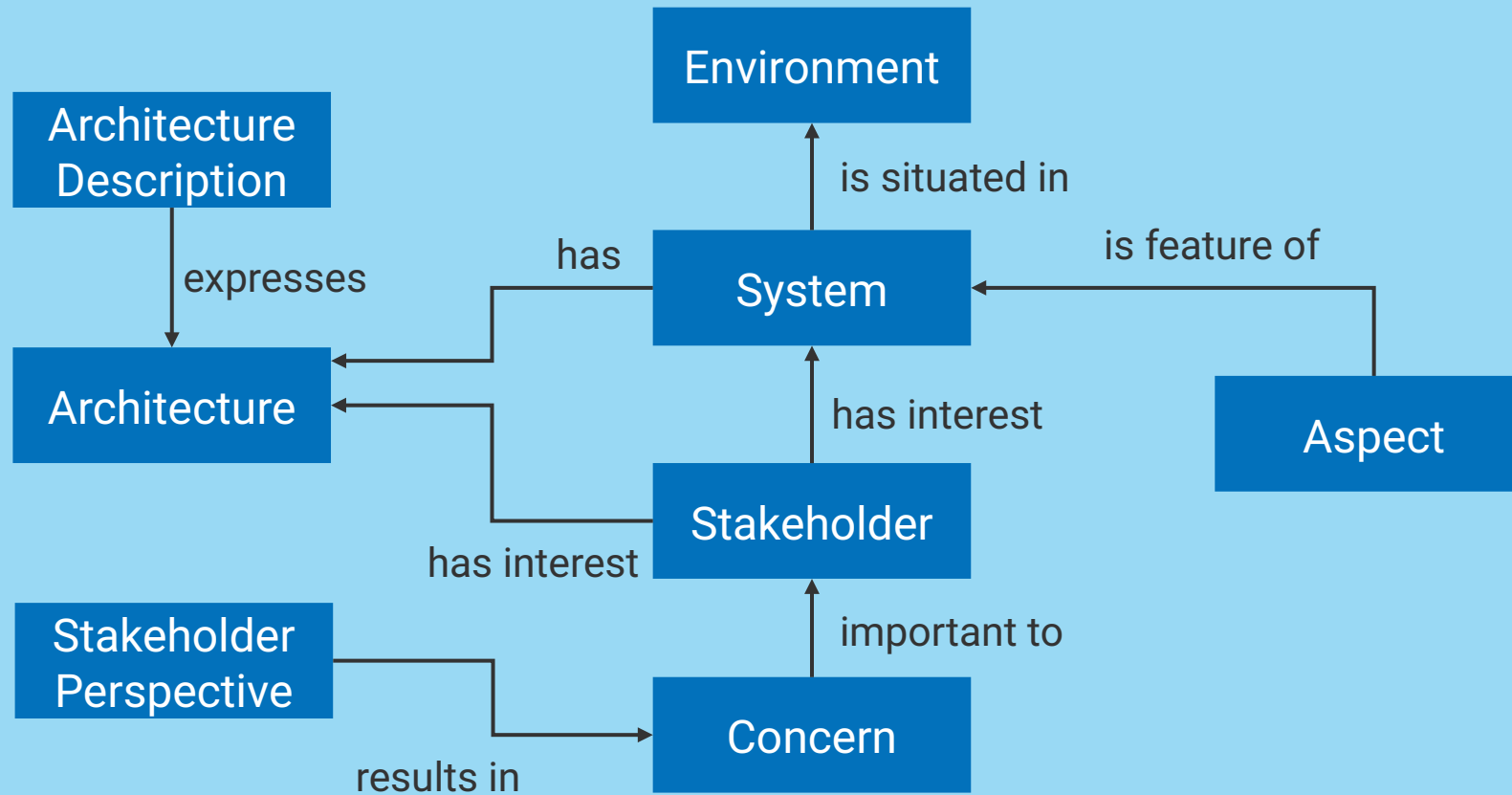
Architecture Modelling

UML

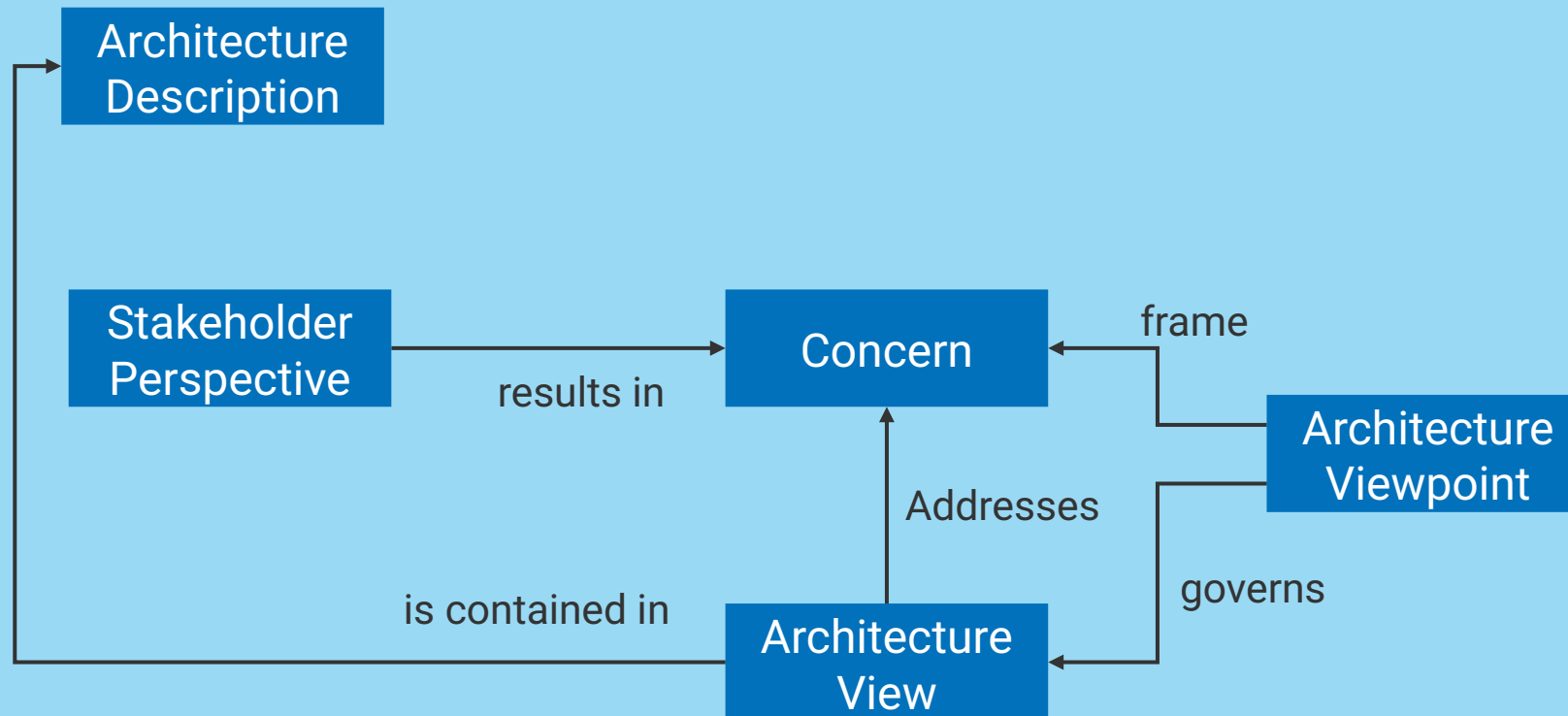
Architecture Description in ISO/IEC/IEEE 42010 (1/2)



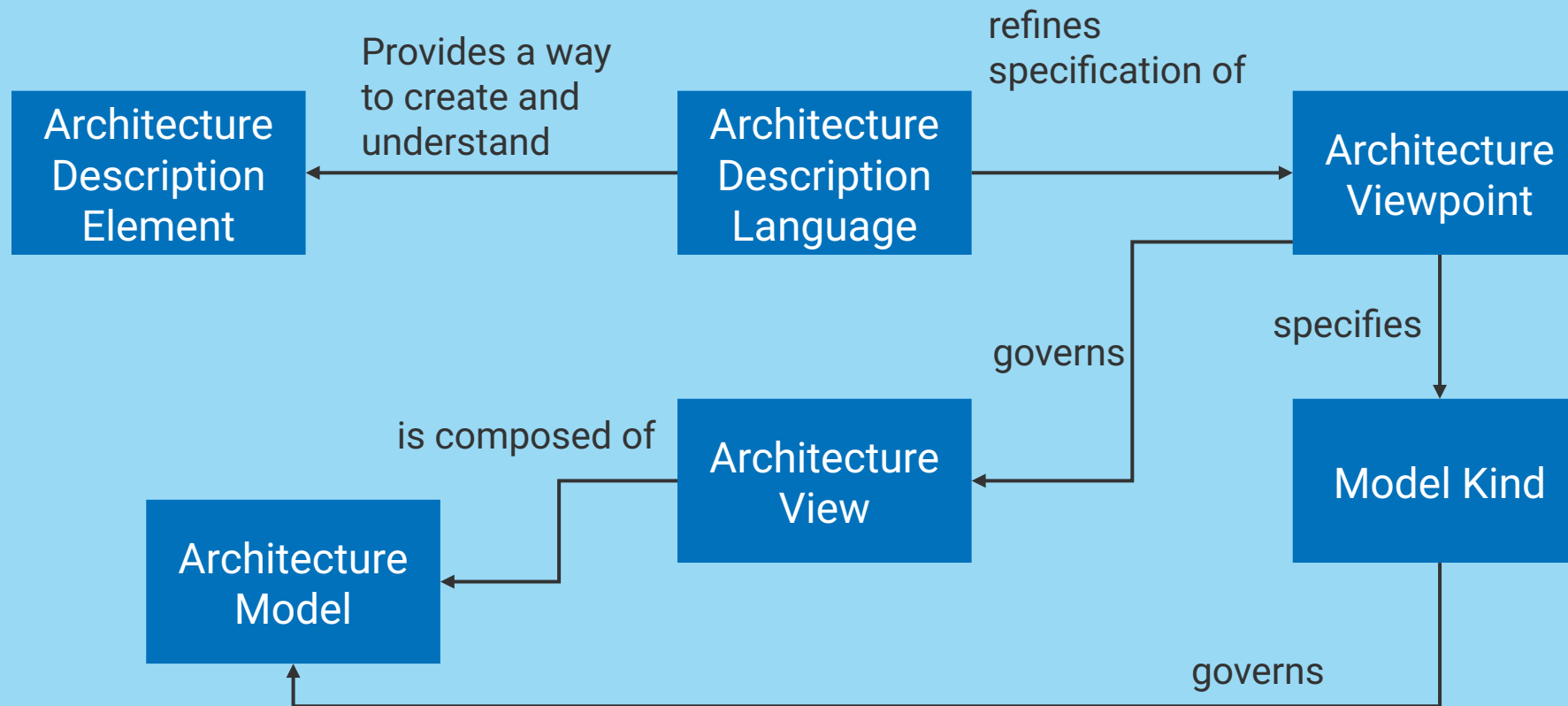
Architecture Description in ISO/IEC/IEEE 42010 (2/2)



Architecture viewpoints and views in ISO/IEC/IEEE 42010



Architecture Description Language in ISO/IEC/IEEE 42010

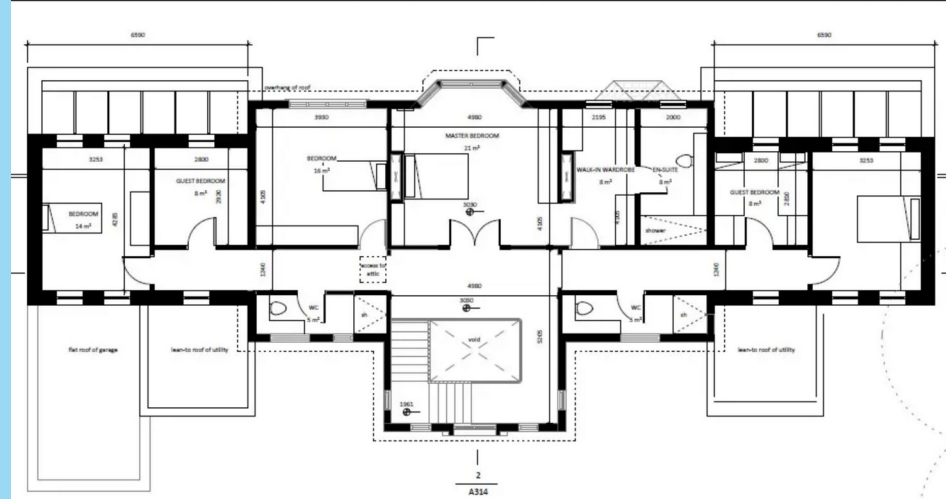


Viewpoints and Views Examples – Building Analogy (1/2)

Viewpoints:

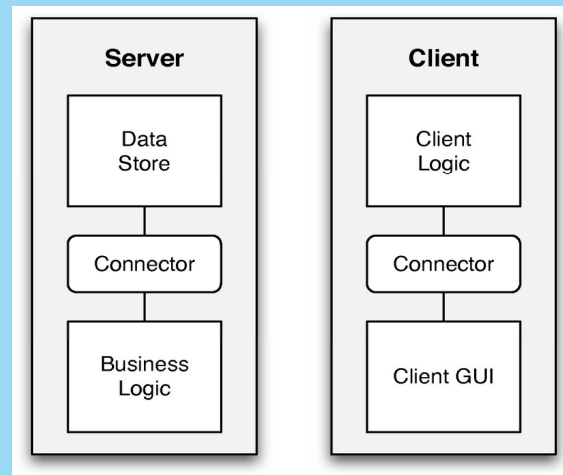
- Interior layout
- Accessibility
- Safety
- Cultural heritage
- Circulation

Viewpoints and Views Examples – Building Analogy (2/2)

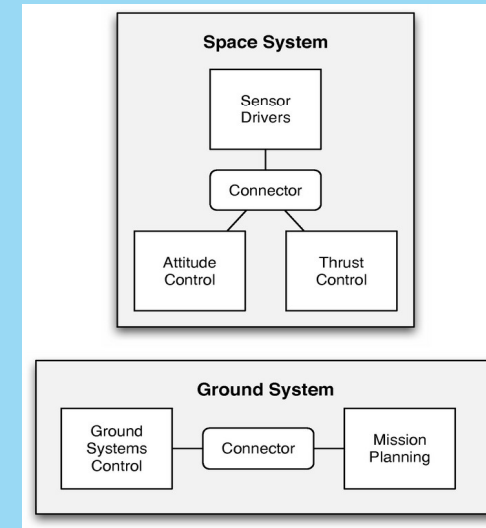


Both instances of the
(building) Interior layout
viewpoint

Viewpoints and Views Examples



Deployment view of a 3-tier application



Deployment view of a Lunar Lander system

Both instances of the deployment viewpoint

Architecture Description (Len Bass et al.) Viewtype $\approx$ IEEE Viewpoint

A software architecture should describe at least three different aspects/perspectives of software

- **Module Viewtype:** Structure as a set of implementation units
- **Component-and-Connector Viewtype:** Structure as a set of elements with run-time behaviour and interactions
- **Allocation Viewtype:** Relation to non-software elements in its environment (development & deployment)

Architecture Description (Len Bass et al.) Viewtype $\approx$ IEEE Viewpoint

Module Viewtype

- What is the primary functional responsibility?
- What other software elements is the module allowed to use?
- What are the relationships?
Generalization? Specialization?

Component and Connector Viewtype

- What are the major executable components and how do they interact?
- Can the system reconfigure itself? If yes, how?
- How does data flow and is being stored

Allocation

- How are software elements assigned to physical resources?
- How is the assignment of software elements to development teams?

Commonly-Used Architectural Viewpoints

Commonly-Used Architectural Viewpoints

- Capture the logical (often software) entities in a system and how they are interconnected.

Physical Viewpoints

- Capture the physical (often hardware) entities in a system and how they are interconnected.

Deployment Viewpoints

- Capture how logical entities are mapped onto physical entities.

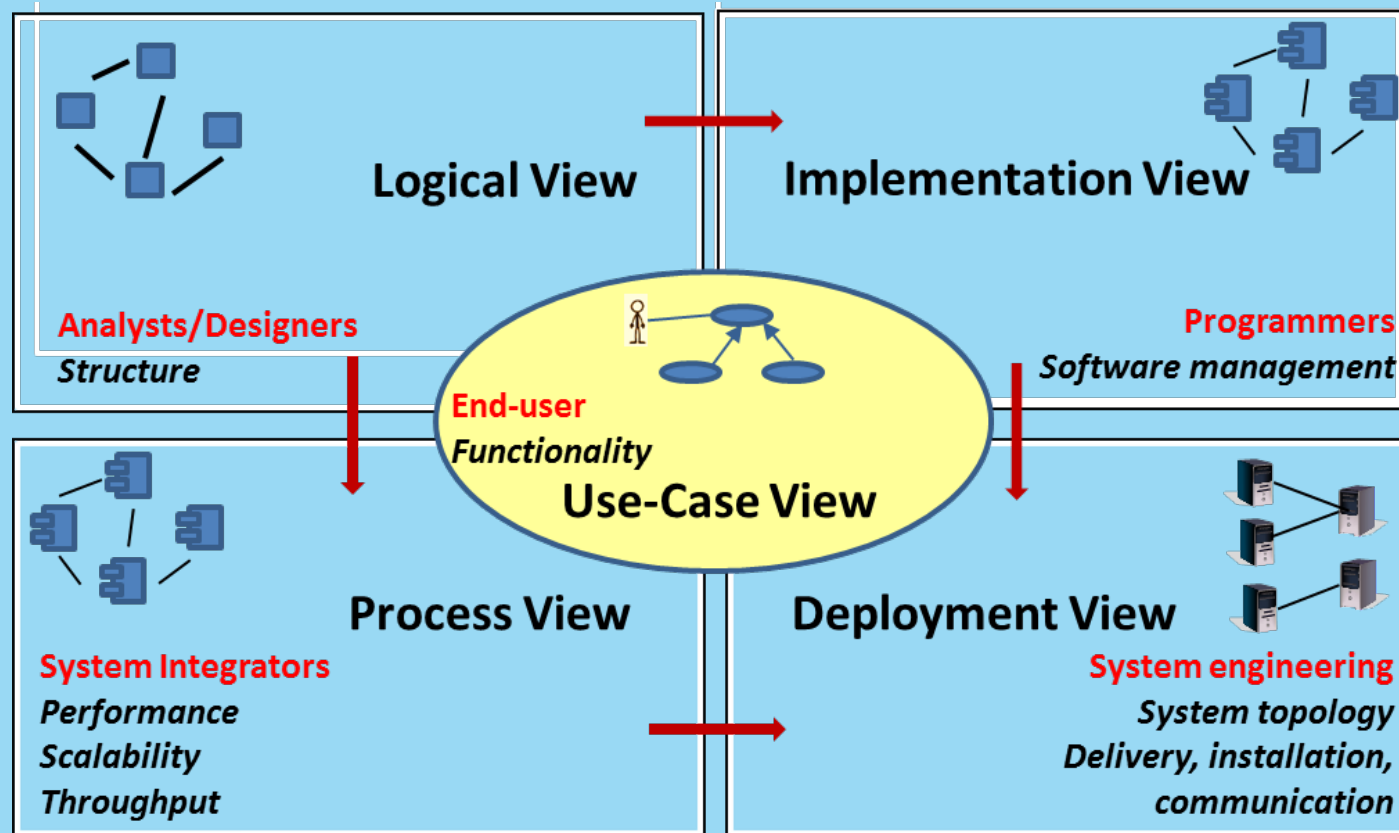
Concurrency Viewpoints

- Capture how concurrency and threading will be managed in a system.

Behavioral Viewpoints

- Capture the expected behavior of (parts of) a system.

Phillippe Kruchten: 4+1 View



Which Architectural Views (of Which Viewpoints) are Relevant?

Key message:

- No general set of views that adequately describe architectures of software-intensive systems
 - Architects need to select what is useful for analysis or communication
- **Selection of views depends on**
 - Which kind of system the architecture describes
 - Who the stakeholders are
 - What concerns the stakeholders have
 - How they will use the documentation
 - Which analyses should be possible
- **Challenge:** Consistency among views

Software Architecture: Communicating Architecture

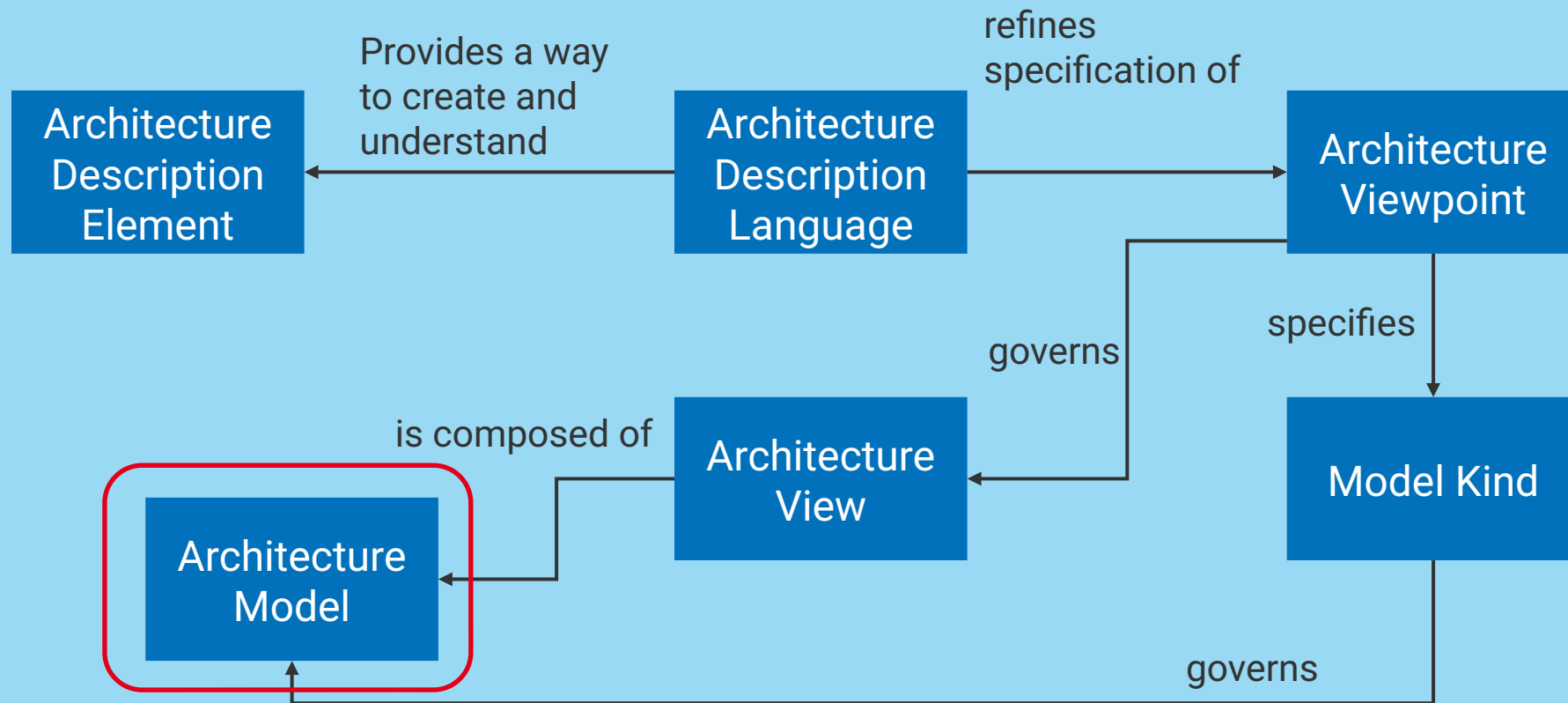


Architecture Description

Architecture Modelling

UML

Architecture Description Language in ISO/IEC/IEEE 42010



Why do we model?

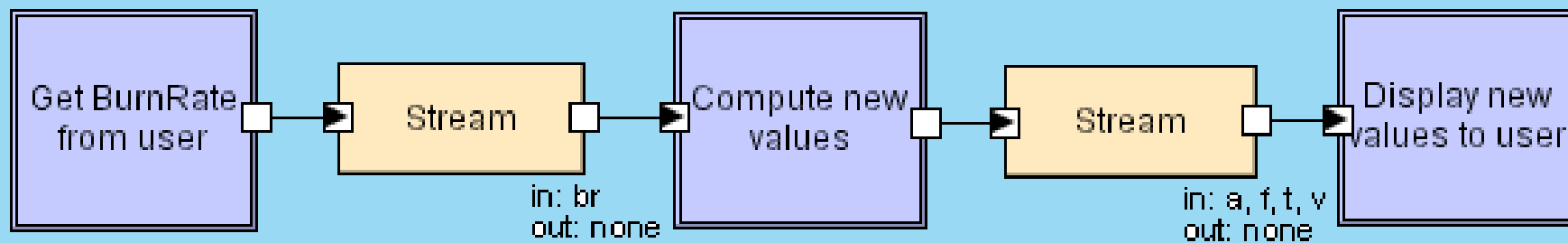
Why do we model?

- Communication
- Documentation
- Earlier fault detection
- Lower change and costs
- Generation of other artifacts
- ...

What do we model? (1/4)

Basic architectural elements

- Components
- Connectors
- Interfaces
- Configurations
- Rationale – reasoning behind decisions



What do we model? (2/4)

Elements of the architectural style

- Inclusion of specific basic elements (e.g., components, connectors, interfaces)
- Component, connector, and interface types
- Interaction constraints
- Behavioral constraints
- Concurrency constraints
- ...

What do we model? (3/4)

Static and Dynamic Aspects

- **Static** aspects of a system do not change as a system runs
 - e.g., topologies, assignment of components/connectors to hosts, ...
- **Dynamic** aspects do change as a system runs
 - e.g., State of individual components or connectors, state of a data flow through a system, ...
- This line is often unclear
 - Consider a system whose topology is relatively stable but changes several times during system startup

Important distinction between:

- Models of dynamic aspects of a system (models do not change)
- Dynamic models (the models themselves change)

What do we model? (4/4)

Functional and **non-functional** aspects of a system

- **Functional**
 - “The system prints medical records”
- **Non-Functional (Quality)**
 - “The system prints medical records *quickly* and *confidentially*”
- Architectural models tend to be functional, but like rationale it is often important to capture non-functional decisions even if they cannot be automatically or deterministically interpreted or analyzed

Architecture Description Language (ADL)

„ A language (graphical, textual, or both) for describing a software system in terms of its architectural elements and the relationships among them.”

Clements et al., 2002

Taylor et al.'s (broad) view on ADLs

- textual or graphical,
- informal (e.g., PowerPoint), semi-formal, formal,
- domain-specific or general-purpose,
- proprietary or standardized, ...

Surveying Modeling Approaches (Partly Historic)

Generic approaches

- Natural language
- PowerPoint-style modeling
- UML, the Unified Modeling Language
- PlantUML

Early ADLs

- Darwin
- Rapide
- Wright

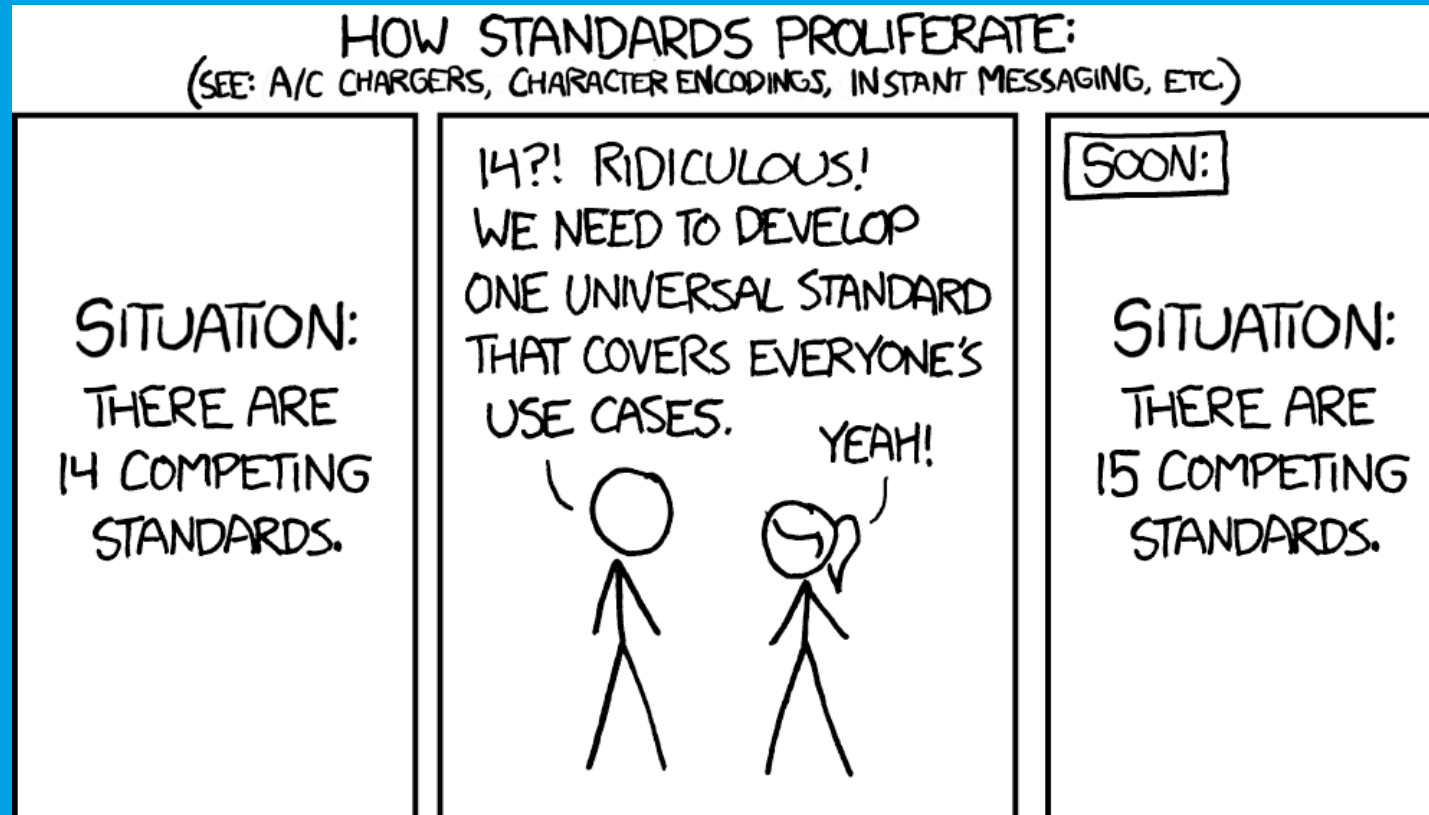
Domain- and style-specific languages

- Koala
- Weaves
- AADL
- Palladio

Extensible ADLs

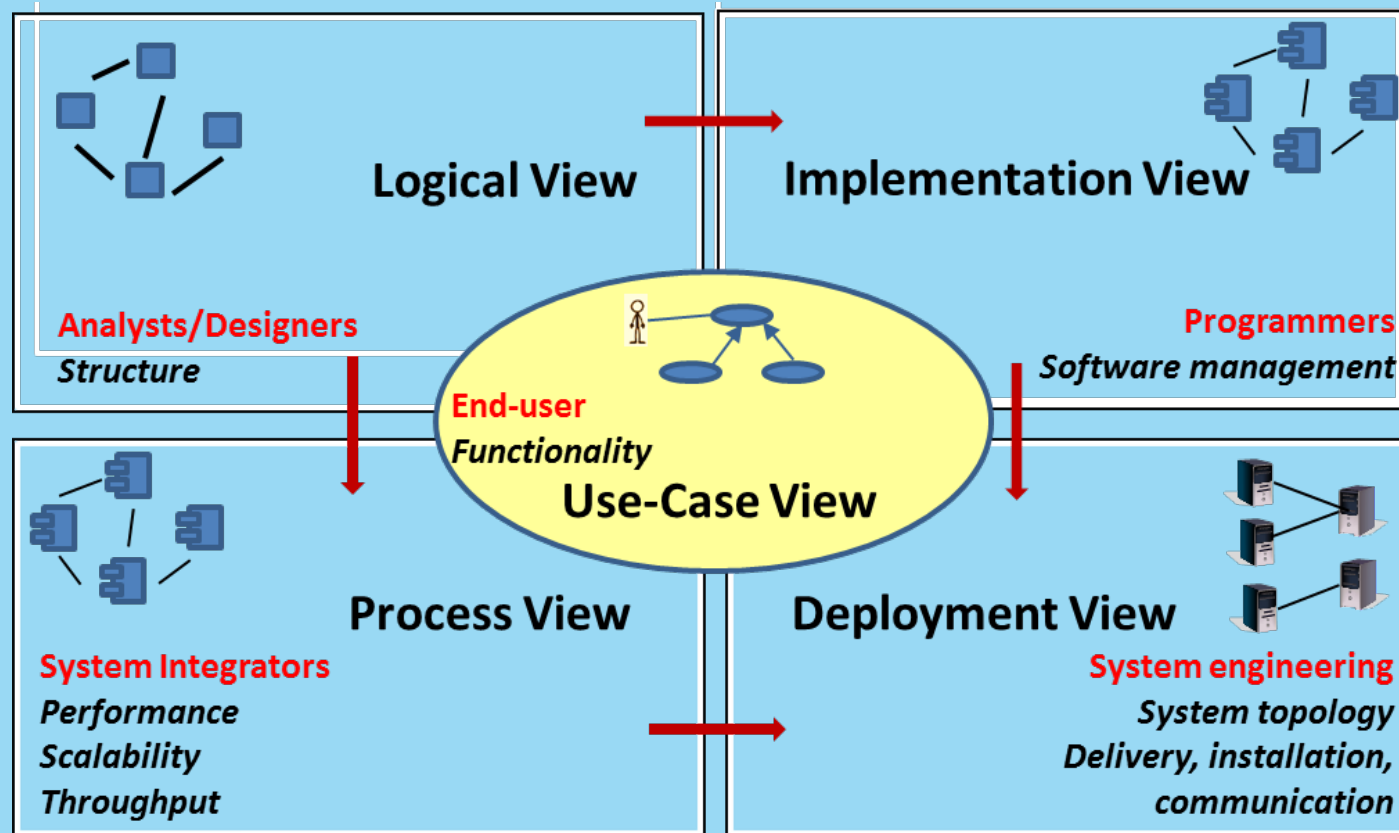
- Acme
- ADML
- xADL

UML – The Unified Modeling Language



<https://xkcd.com/927>

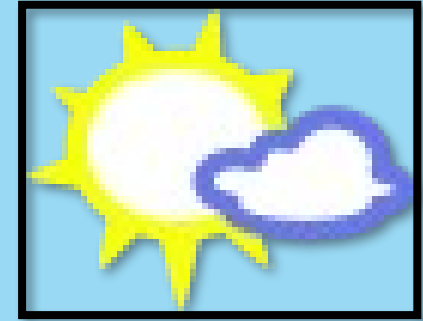
Phillippe Kruchten: 4+1 View



Running Examples: Sunshine2Go + WolfDesk

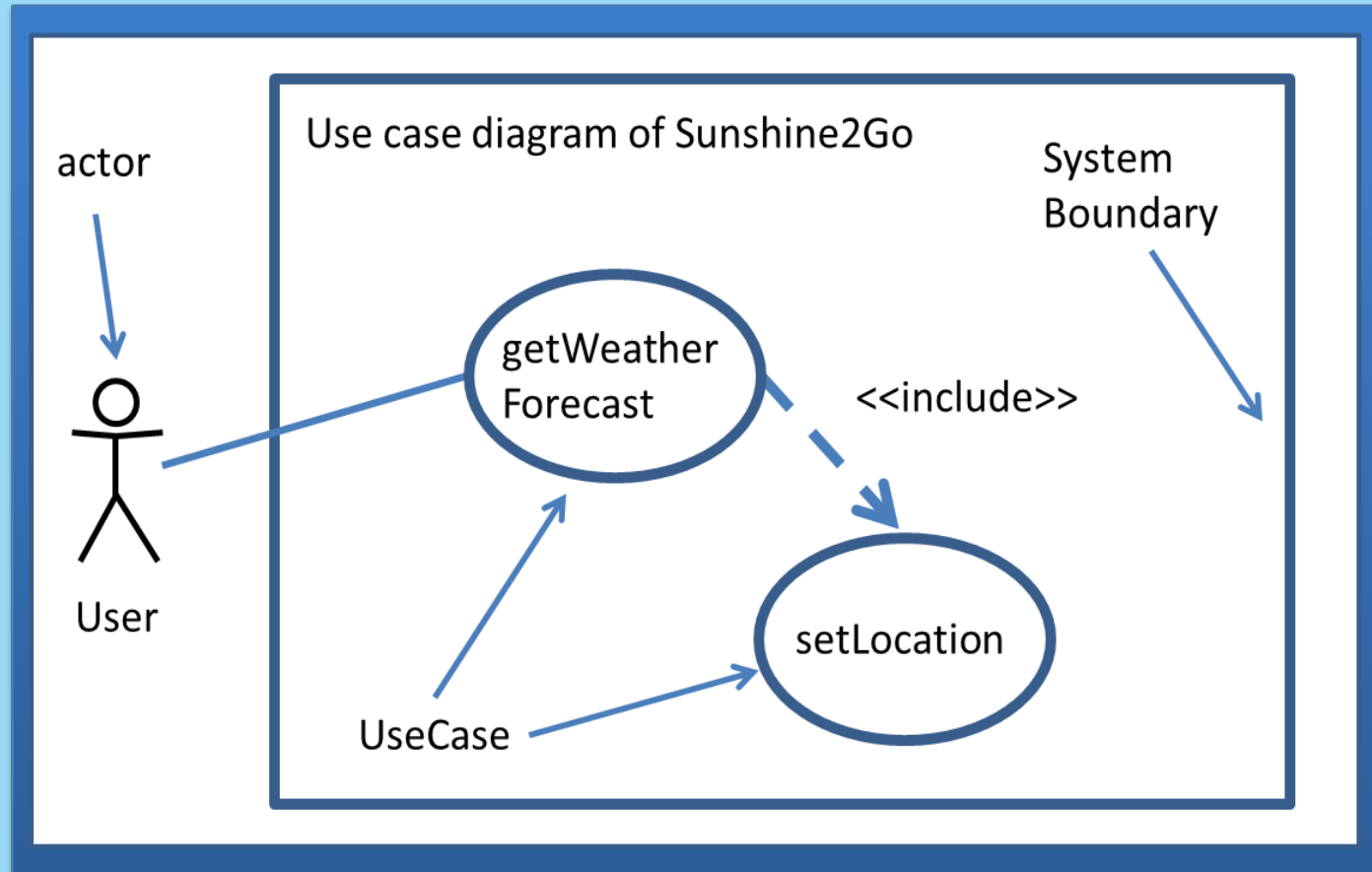
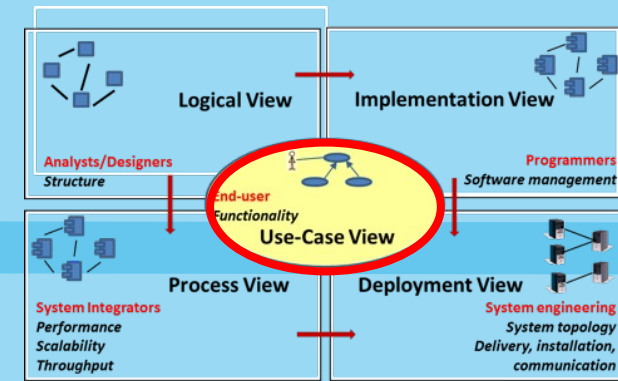
Web service examples:

- Sunshine2Go
 - querying the weather forecast of the next 7 days
 - reports weather conditions of next 7 days for given location in the world
- WolfDesk
 - Support ticket system



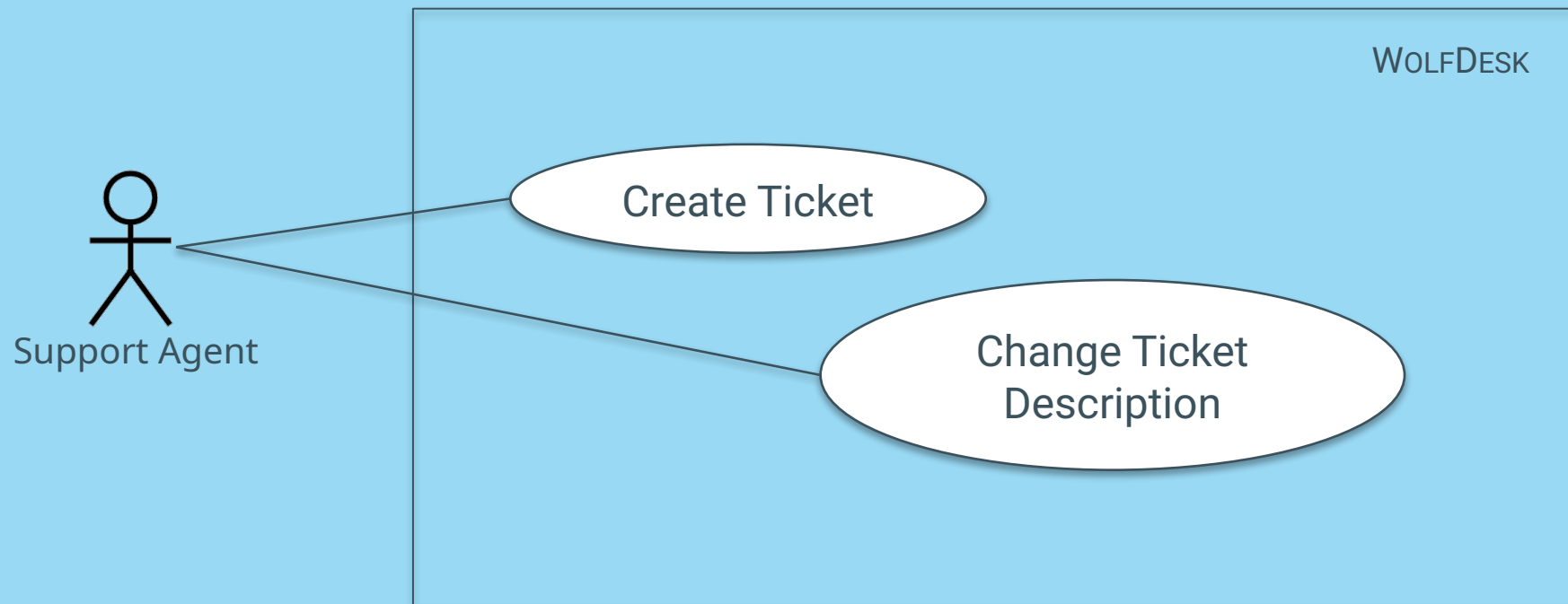
What is the goal of a Use Case Diagram?

UML Use Case Diagram

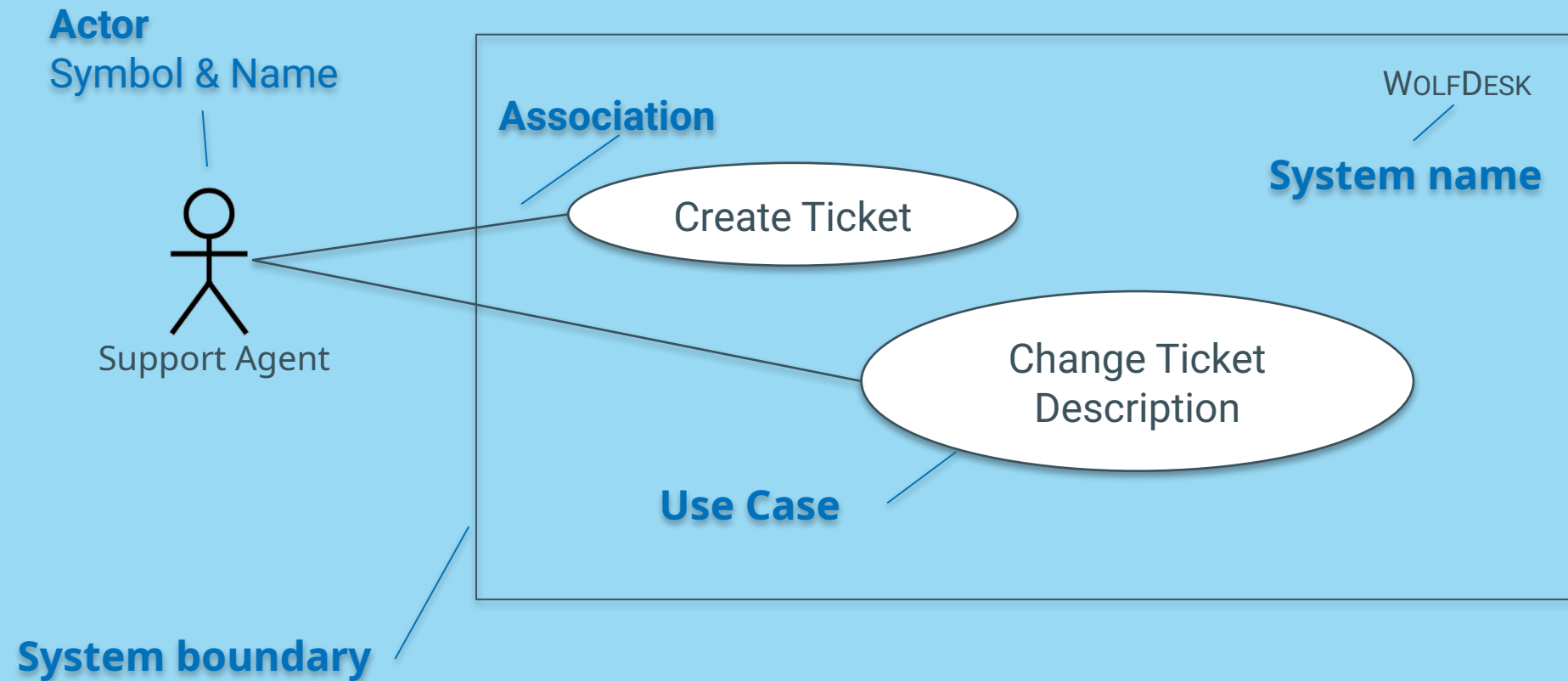


OMG Standard

UML Use Case Diagram – WolfDesk



UML Use Case Diagram – WolfDesk

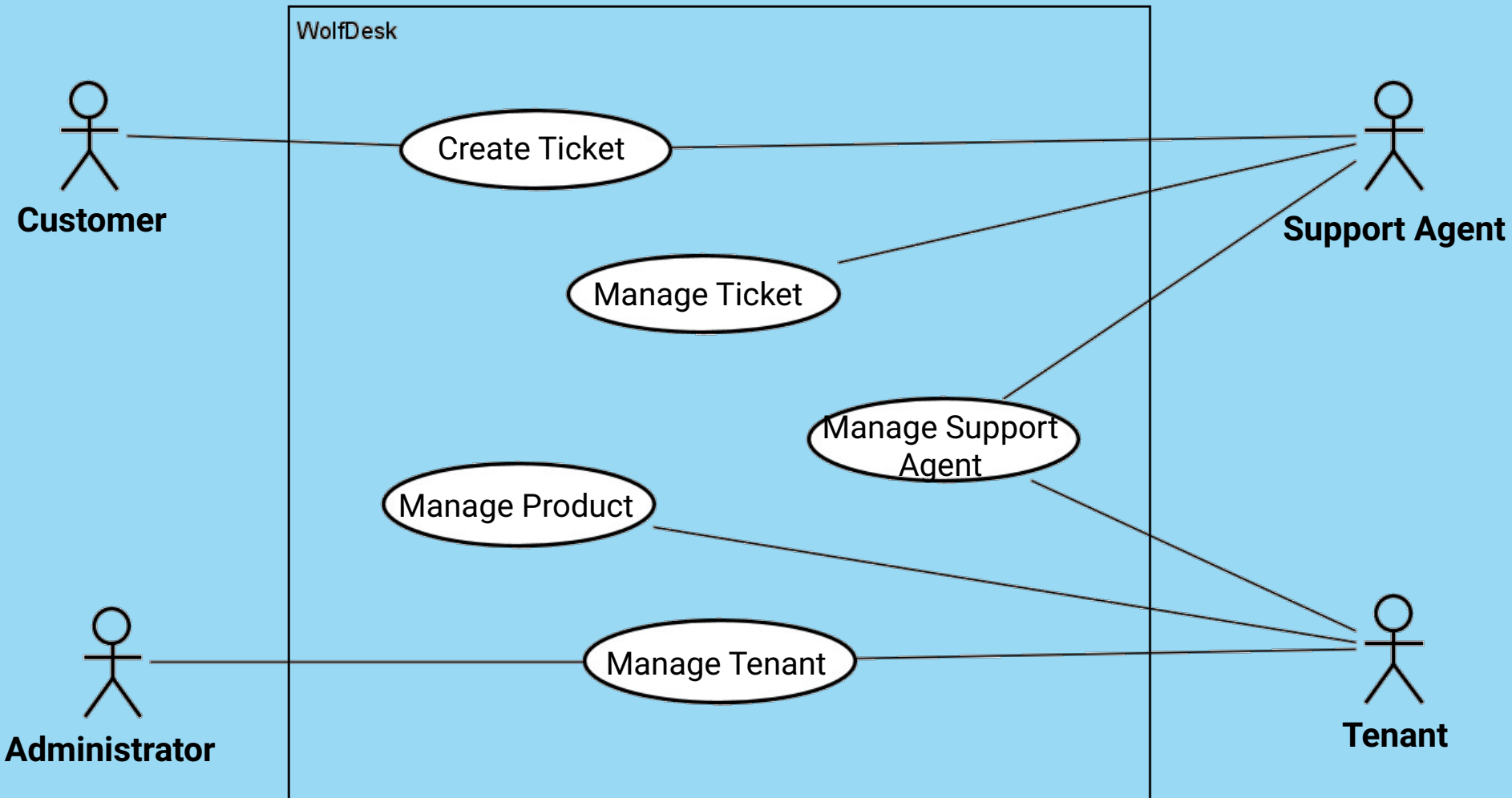


UML Use Case Diagram – Association

- Relationship between actor and use case
- Actor is involved in the use case (active/triggering or passive)
- Several actors can be involved in a use case

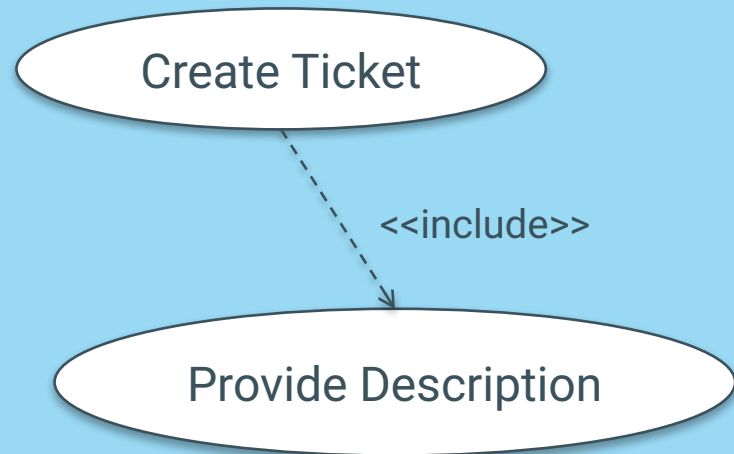


UML Use Case Diagram – Multiple Actors



- One of many Use-Case diagrams for this system
- Very course grained with 5 use cases only
- Fokus on overview, not details

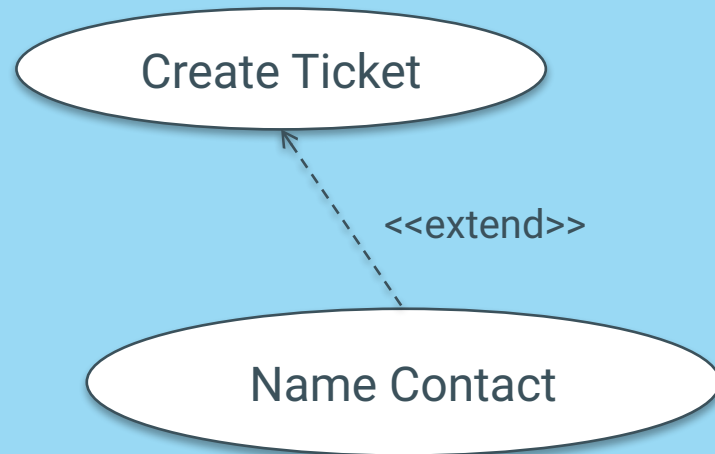
Use Case Diagram – Further Relations (1/3)



Inclusion

- Relationship between two Use-Cases
- When “*Create Ticket*” is executed then “*Provide Description*” is also performed.

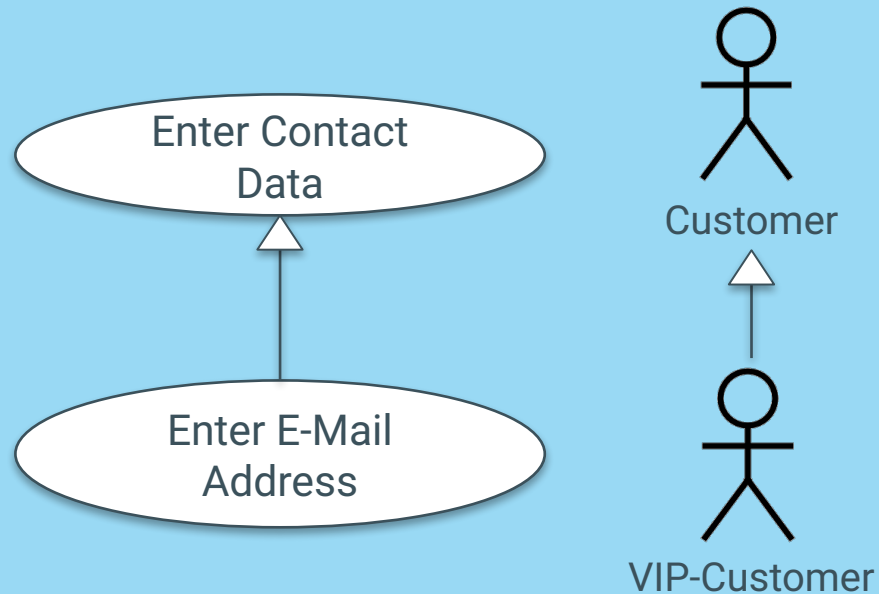
Use Case Diagram – Further Relations (2/3)



Extension

- Relationship between two Use-Cases
- In some cases of *"Create Ticket"* *"Name Contact"* is also executed.
- Often controlled using a condition or manually

Use Case Diagram – Further Relations (3/3)

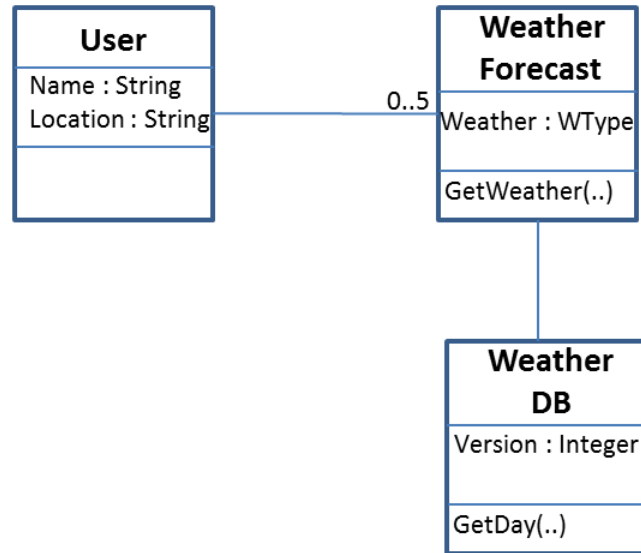
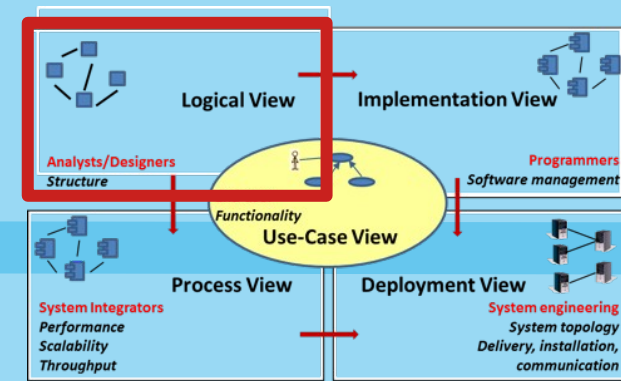


Generalization

- Relationship between two Use-Cases
- Case/Actor is a special variant of more general case/actor
- Case/Actor Often fulfills properties of more general case/actor

What is the goal of a Class Diagram?

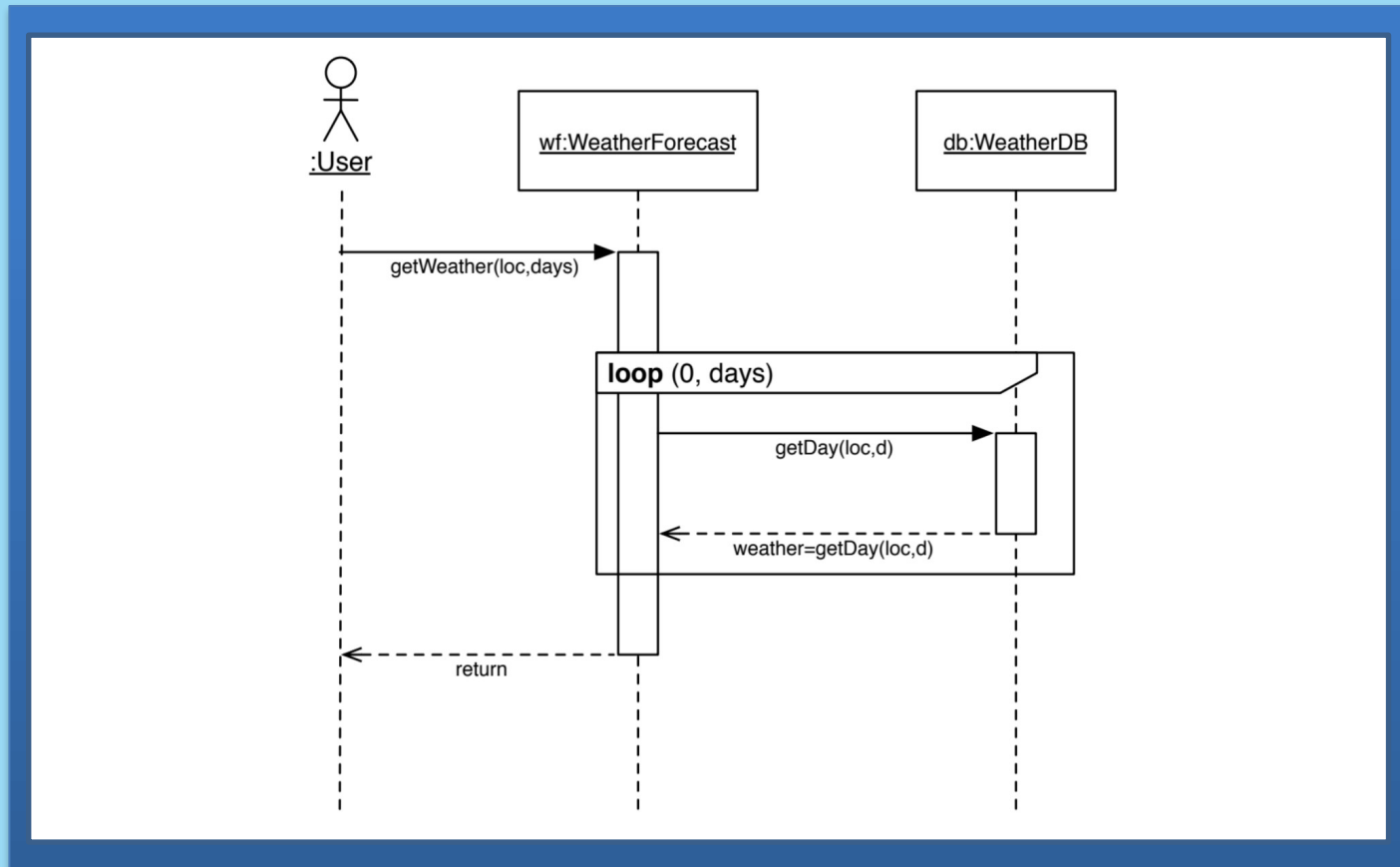
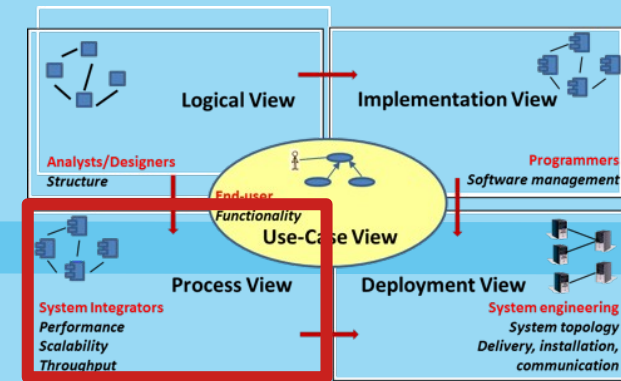
UML Use Case Diagram



OMG Standard

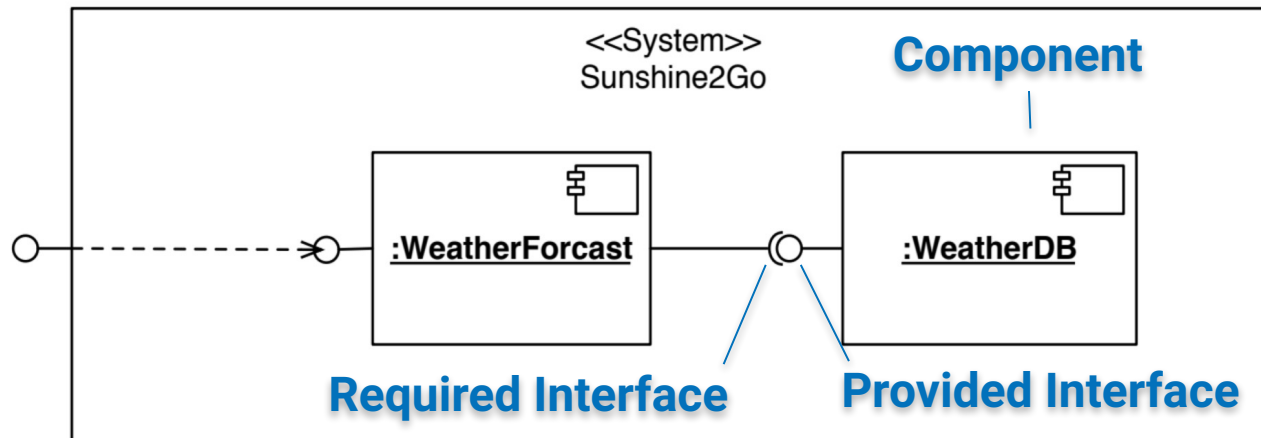
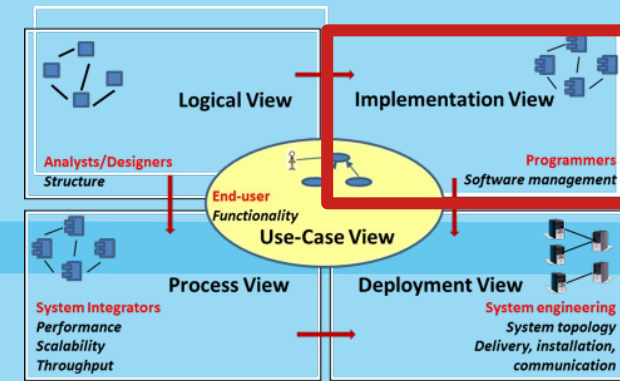
What are some common pitfalls with the UML Class Diagram?

UML Sequence Diagram



OMG Standard

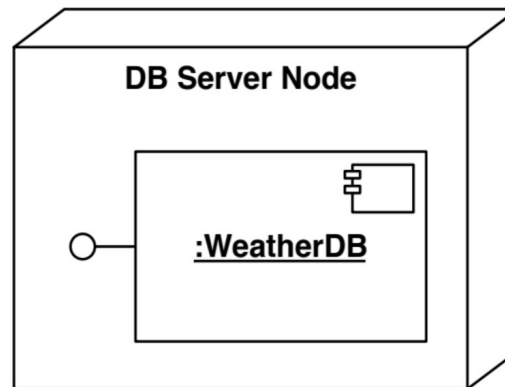
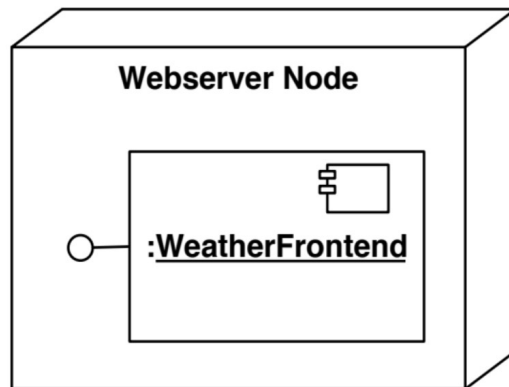
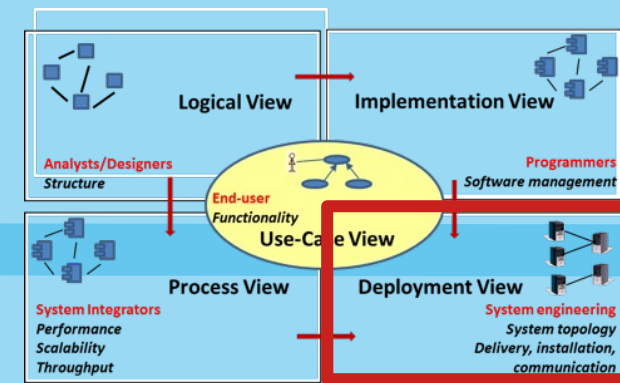
UML Component Diagram



OMG Standard

What is the difference between the Component and Class Diagram?

UML Deployment Diagram



OMG Standard

Penultimate Question

What is a “Domain Model”?

Domain Model

A model modeling the (business) domain.

NOT: A model modeling our software!

Final Question



Is this a domain model? (POS example)

A class-diagram for our POS software-architecture



A sequence diagram showing interaction between cashier and customer



A sequence-diagram detailing how cash transactions are stored in our database



A class diagram representing the properties of items that can be scanned by our cash register



Task – Domain Model (Class Diagram)

Window Blind Controls

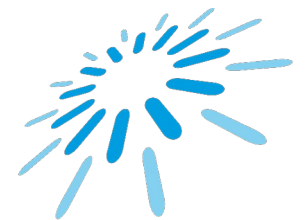
Model a Window Blind Control.

**It controls Window Blinds for multiple windows.
Blinds can be moved up/ down and they can be tilted.
Blinds consist of multiple segments and control
threads.**



Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences